

ros2_control Concepts & Simulation

Solving the problem EVERY robot has (with ros2_control)

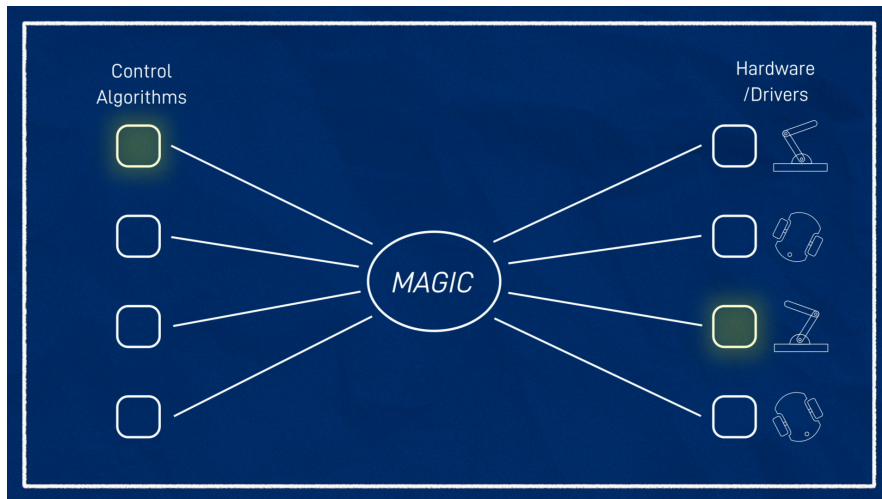


Why do we need ros2_control?

Today we're going to learn how to solve a problem that literally every robot has, and that's *control*. A core part of every robot is to take in some kind of input (from an operator or the environment), do some thinking, and drive an actuator - whether that's a motor, a hydraulic system, or something else.

There are many different kinds of actuators and interfaces out there, and many different control methodologies that we use to solve problems with them. But within this huge universe of options, there's actually a lot of commonality, and if we don't have some kind of standardised system - if we're just writing a custom controller and custom interface for every application - we end up wasting time rewriting things and solving problems that people have solved hundreds of times before

Instead what we want is a framework where we can write the drivers for different hardware platforms, and the algorithms for different control methodologies, and have these speak a common language so we can pick and choose what we need.



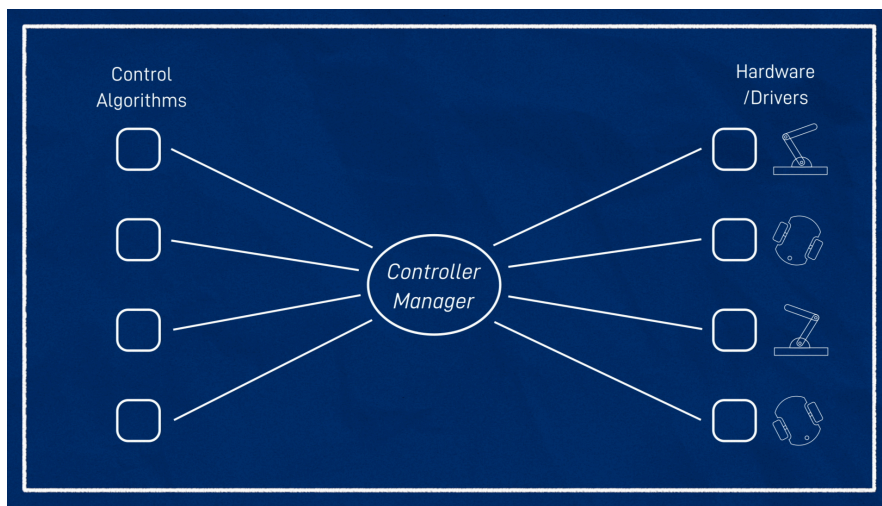
You might think that this is a problem well suited to ROS topics - just create some standard messages, write nodes for the controllers and drivers, and you're done! One problem with this approach is that although topics are generally pretty fast, we want something like this to run *really* fast, there should be no delay between the controller and the driver. That's where ros2_control comes in.

The ros2_control framework is a powerful and complex beast, so to understand it we're going to have to take it step by step. It might be tempting to give up and say "it would be quicker to just write this from scratch" and although maybe it would be (probably not), the power of ros2_control is not just that it can make individual projects simpler, but that it makes reusing code, upgrading, and modifying systems *way* easier.

In this tutorial we'll look at how ros2_control works, and how to use it in a Gazebo simulation, and then in the next tutorial we'll get it up and running on a real robot so we can drive it around.

Controller manager

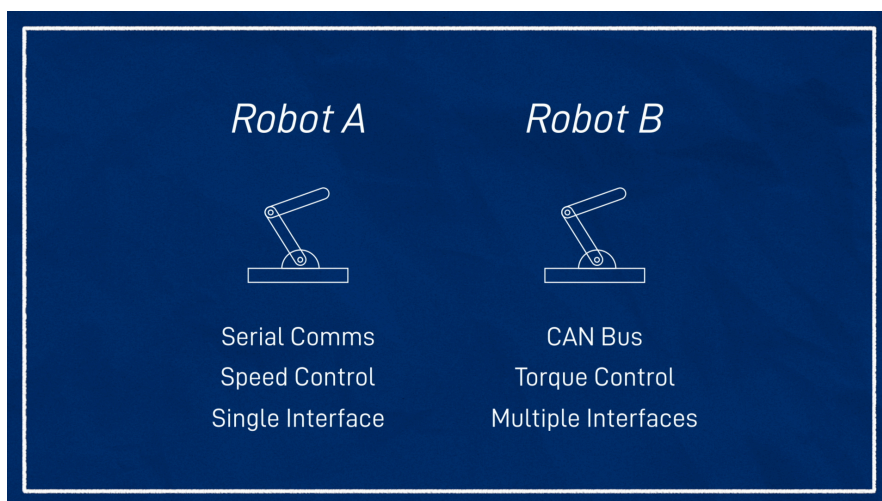
At the centre of it all we have something called the controller manager. It's going to find all the bits of code for our hardware drivers and our controllers and link them together. To achieve this, it uses a plugin system - so each of these things isn't running its own executable, it's just a library that is loaded up at runtime and has a set of functions that will link in to the system.



Hardware Interfaces

Let's take a look at the hardware side first.

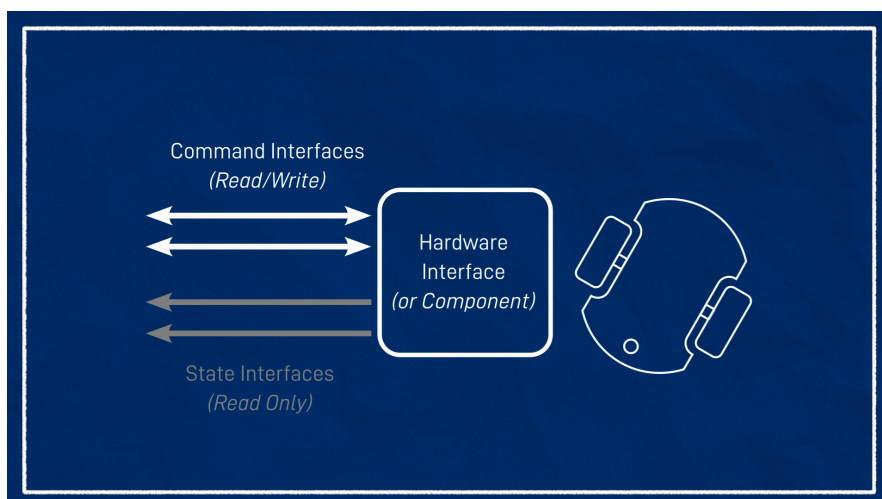
Different hardware needs to be controlled differently. You might have two very similar-looking robots, but one's motors speak via serial, and the other via CAN bus. One might let you control speed and position, the other only speed, or speed and torque. One might have a single comms interface for all motors, the other might have an interface per motor.



Whatever our hardware looks like, to use `ros2_control` with it, we need something called a *hardware interface* (sometimes called a *hardware component*). This is a bit of code that talks to the hardware and exposes it in the `ros2_control` standard way. Ideally if you've bought a robot, the creator or someone else online will have written the hardware interface for you.

If you're following along with my build series you'll be able to use the one I've written, but otherwise you might need to write one yourself. I may cover that in a future tutorial, but it's not currently planned.

The hardware interface acts as an abstraction so that as users, all we need to understand is the way it represents our hardware, which is through *command interfaces* and *state interfaces*. Command interfaces are things we can control, and state interfaces are things we can only monitor.



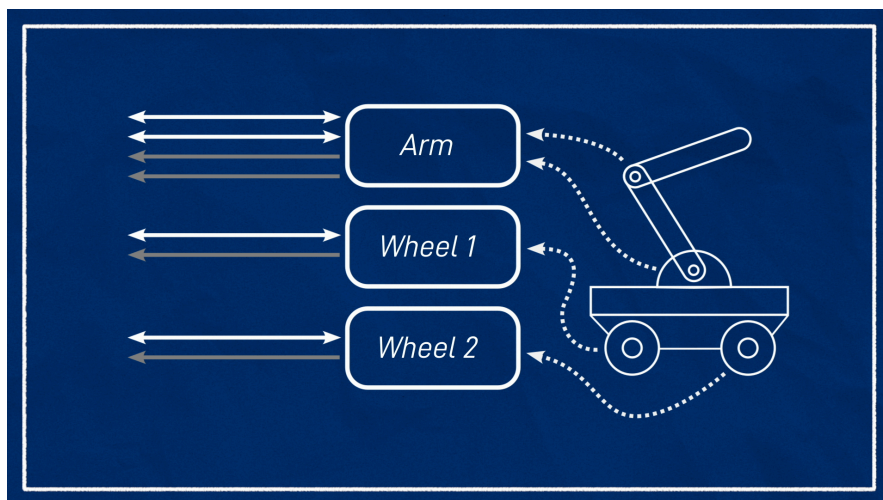
As an example, on the robot I'm building the only thing we can control is the velocity of the motors, so my hardware interface will have two command interfaces, both velocity control, one for each motor. And what state interfaces will it have? Well, using the encoders we can measure both the velocity AND

the position of the wheels. So it will have four state interfaces, position and velocity for each motor. If we had other things to measure like torque or current (or even a non-motor sensor like battery level) these could be added in too.

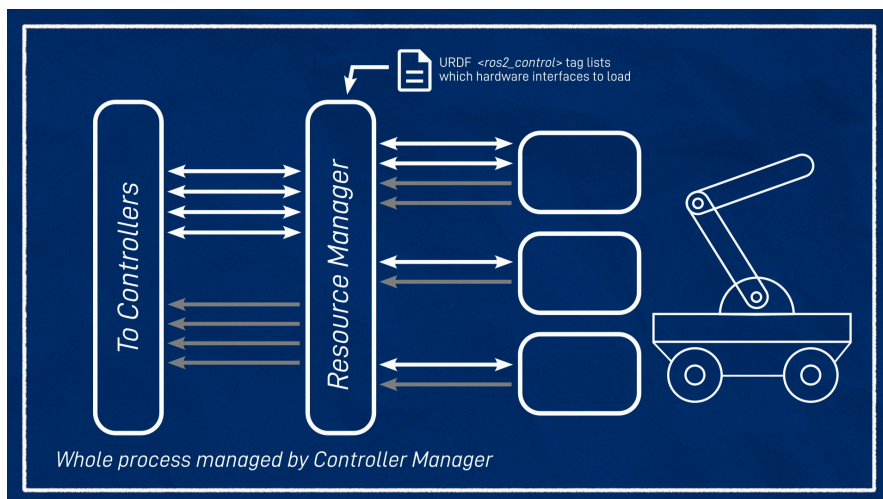
Below is a preview of what we'll see later on. With the hardware interface loaded, we can see the two command interfaces and the four state interfaces. At this level of abstraction, there is really no understanding of what the robot is. It doesn't matter if this is a two-wheeled diff drive robot, or a 2 degree of freedom robotic arm that is for some reason only speed controlled - all we see are these six interfaces.

```
dev@dev-PC:~/dev_ws$ ros2 control listHardwareInterfaces
command interfaces
  left_wheel_joint/velocity [unclaimed]
  right_wheel_joint/velocity [unclaimed]
state interfaces
  left_wheel_joint/position
  left_wheel_joint/velocity
  right_wheel_joint/position
  right_wheel_joint/velocity
dev@dev-PC:~/dev_ws$
```

One robot may have multiple hardware interfaces, each of which can have multiple command and state interfaces. There are many reasons you might have this - perhaps you have a robot arm mounted on a mobile base, each with separate motor control systems, or perhaps each of your motors is fully independently controlled with their own driver instance.



To deal with this, the controller manager uses something called the *resource manager* which gathers up all the hardware interfaces and expose them all together, so the controllers just see a big list of command and state interfaces.

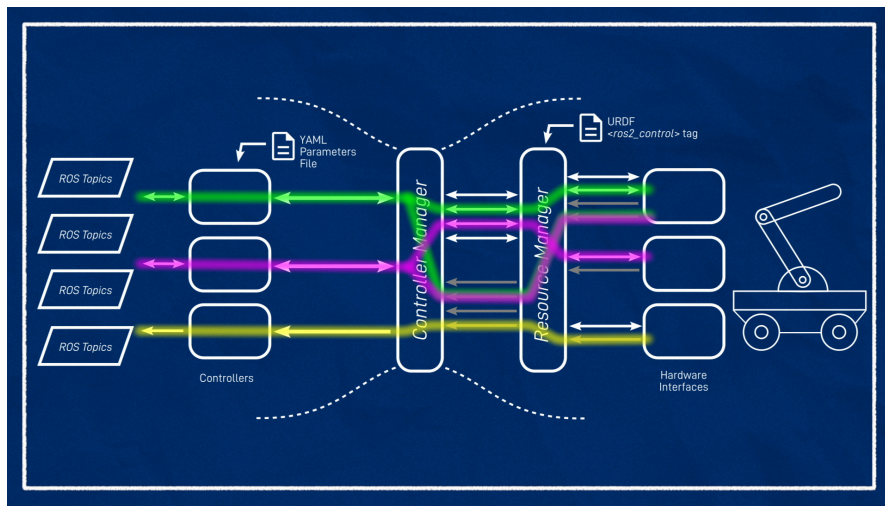


And how does the controller/resource manager know about the hardware interfaces? Because they're tied pretty closely to the robot's hardware design, we put it into the URDF using a `<ros2_control>` tag. We'll take a closer look at this later.

So that's the hardware interfaces, what about the controllers?

Controllers

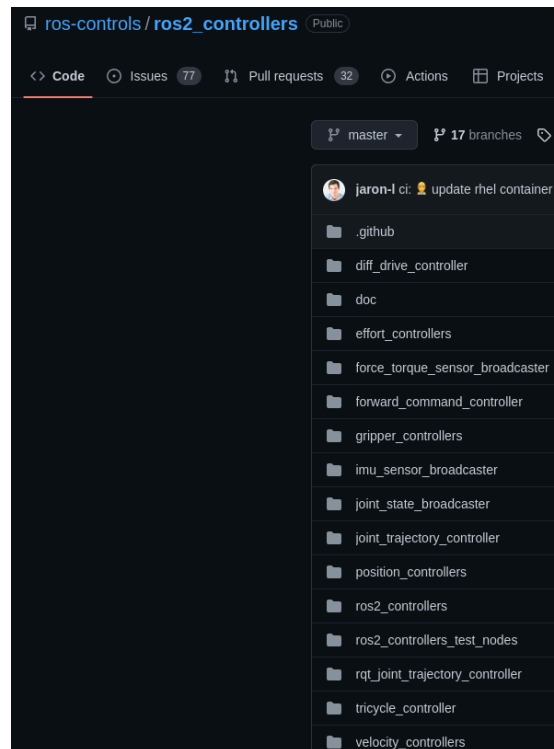
The controllers are how the rest of the ROS ecosystem interacts with `ros2_control`. On one end they'll be listening to a ROS topic for control input - could be joint positions or mobile body velocities - they take this input and use some kind of algorithm to figure out the appropriate actuator speeds, positions, etc, and send that to the appropriate hardware interfaces. They can also publish to ROS topics for command feedback or state information they receive from the hardware, and a single controller can pass information in either or both directions. This last point can be a little confusing - it's common to have a controller that doesn't actually "control" anything, it just publishes sensor/feedback data.



Here the controller manager's job is to take the controllers it's asked to load, and match them up with the right command and state interfaces that the resource manager is exposing. To set up the controllers we write a YAML file with the various parameters we need, and pass that in to the controller manager. Once this is all loaded up, we can tell it to start and stop the controllers as needed.

Just like we can have multiple hardware interfaces, we can also have multiple controllers in one robot, as long as they don't both want to claim the same *resources*, they're not trying to command the same (or similar) interface. They can share state interfaces though, since they're read-only.

While the hardware interfaces are going to be made specifically for different robot *hardware*, the controllers are going to be dependent on the robot *application*, and since there are a few common applications, the `ros2_controllers` package provides a bunch of controllers that should cover most people's needs. The project we are working on in these tutorials is a differential drive robot, so naturally we'll be using the `diff_drive_controller`, but of course a robot arm or something else will require a different controller. And if there isn't an appropriate controller available, we can write our own!



Note that while the primary controller we'll be using in this project is the `diff_drive_controller`, we'll actually be using a second controller - a `joint_state_broadcaster`. This controller is one of those "non-controlling" ones and is used to ensure our wheel transforms are generated correctly. More on this later.

Running the controller manager

Ok, so we know we've got our hardware interfaces on one side, our controllers on the other side, and a controller manager in the middle. So how do we run this thing?

There are kind of two ways to run a controller manager. The normal (and simpler) way to do it is with the `ros2_control_node` provided by the `controller_manager` package, but you can also write your own node and instantiate the controller manager inside it. We'll get to see a glimpse of that soon, because the `gazebo_ros2_control` plugin runs its own controller manager, but when we run it on our robot in the next tutorial we'll learn how to use the standalone one. Either way, we need to provide it with the details of the hardware interfaces (usually via URDF) and the controllers (usually via YAML parameters).

Then, once the controller manager is running, we need to interact with it to do things like checking the hardware interfaces and starting the controllers. There are a few different ways we can do these interactions:

- It exposes some services that we can call
- It provides the `ros2 control` command line tool to simplify calling the services

- It provides some nodes which will also call the services when they are executed

Using these tools we can start and stop and reconfigure the controllers. Each of methods has its pros and cons which we'll briefly discuss when we use them later in the tutorial.

Testing it out in simulation

So that's the theory, let's start putting it into practice by upgrading our Gazebo robot simulation to use `ros2_control`!

Installing dependencies

First, we need to install some packages. We want:

```
sudo apt install ros-foxy-ros2-control ros-foxy-ros2-controllers ros-foxy-gazebo-ros2-control
```

Update URDF

Now let's update our URDF - I'll be continuing the example URDF from the rest of this tutorial series, but it shouldn't be too hard to follow along if you're just joining in.

You might remember that we had a file called `gazebo_control.xacro` which handled the gazebo control code. We're going to create a new file called `ros2_control.xacro` and then in our `robot.urdf.xacro`, we'll comment out the old one and add in the new one.

```
<!-- <xacro:include filename="gazebo_control.xacro" /> -->
<xacro:include filename="ros2_control.xacro" />
```

Make sure we start by adding our normal `<xml>` and `<robot>` tags, they can be copied from one of the other xacro files.

So what needs to go in this file? For now, we'll have two main tags. There is a `<ros2_control>` tag, which will have the details of the hardware interface (for the controller manager) and then a `<gazebo>` tag which will tell Gazebo to load up the extra code that it needs.

First up we'll fill in the `ros2_control` tag. We need to give it a name, I'll call mine `"GazeboSystem"`, and set the type, which is `"system"`. The other types are `actuator` and `sensor` which can be used when the hardware is a single actuator or sensor, but `system` is the most general.

Inside this, we add a `<hardware>` tag with a `<plugin>` tag inside that, and this is going to be the name of the hardware interface it needs to load up. This is a piece of code that has been installed and registered with ROS separately, that will talk to Gazebo just like a normal hardware interface talks to a motor controller. In this case, the name of the plugin we want to use is `gazebo_ros2_control/GazeboSystem`, so we can put that in there.

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
</ros2_control>
```

Now we need to specify which joints are to be controlled, so we make `<joint>` tags and set the `name` - in this case the joints that go from the base to the left and right wheels. We also need to declare which interfaces are being made available for that joint. As we saw earlier, we're going to have one command interface for each wheel (velocity) with the option of specifying limits, and the two state interfaces (velocity/position).

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
  <joint name="left_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="position" />
    <state_interface name="velocity" />
  </joint>
  <joint name="right_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity" />
    <state_interface name="position" />
  </joint>
</ros2_control>
```

That's the `ros2_control` tag done, we've told it the hardware interface to load and which joints to associate with. Now we need to set up the `<gazebo>` tag.

It begins similarly to `gazebo_control.xacro`, inside our `<gazebo>` tag we have a `<plugin>` tag. This time the plugin file name is `libgazebo_ros2_control.so` and we can give it a name, `gazebo_ros2_control`. This plugin is actually doing a few different things:

- Loading up the stuff needed on the Gazebo end to talk to the hardware interface
- Running a controller manager
- Finding the URDF provided by `robot_state_publisher`

Because it has the controller manager inside it, it still needs a YAML parameters file to know which controllers to load. To tell it, we need to add a tag called `<parameters>` and this will be the path to our YAML file. Since we don't have that file yet, just leave it blank and we'll come back to it soon.

```
<gazebo>
  <plugin filename="libgazebo_ros2_control.so" name="gazebo_ros2_control">
    <parameters>...to be filled in soon!...</parameters>
  </plugin>
</gazebo>
```

Controller Config

Up in our config directory, let's create a new file in our `config` directory called `my_controllers.yaml`. This file will be a fairly typical ROS parameter file, if you're familiar with them.

The first thing we need parameters for is the controller manager. Two simple parameters to set are the `update_rate` which determines the rate the controllers will update at, and `use_sim_time` because we want to use this with a Gazebo simulation.

More confusing are the parameters that determine our controllers. We set the parameter `name` as the name we want to call our controller, then nested under it is the `type` of the controller. We need to make two controllers, a `diff_drive_controller` and a `joint_state_broadcaster` which we'll dive into next. I've named them `diff_cont` and `joint_broad` to keep things shorter.

```
controller_manager:
  ros_parameters:
    update_rate: 30
    use_sim_time: true

diff_cont:
  type: diff_drive_controller/DiffDriveController

joint_broad:
  type: joint_state_broadcaster/JointStateBroadcaster
```

Directly below the `controller_manager` section, we add sections for the parameters of our controllers. The `diff_drive_controller` has many different parameters to explore, but for now we'll just set the ones below. They are mostly self-explanatory except for `use_stamped_vel` which we'll take a closer look at in a future tutorial.

The `joint_state_broadcaster` simply uses the wheel encoder positions to publish the `/joint_states` message that `robot_state_publisher` needs to generate the wheel transforms. For a mobile robot this is not all that important - we care more about the position of the robot in space than exactly how far each wheel has turned - but it is still nice to have. For a robot arm it is much more important. I have left its parameter block commented out as we won't be setting anything.

Note that we aren't setting `use_sim_time` on the controllers - it gets propagated down from the controller manager.

```
## Directly below the controller_manager section

diff_cont:
  ros__parameters:

    publish_rate: 30.0 # You can set this higher than the controller manager update rate, but it will be throttled
    base_frame_id: base_link

    left_wheel_names: ['left_wheel_joint']
    right_wheel_names: ['right_wheel_joint']
    wheel_separation: 0.35
    wheel_radius: 0.05

    use_stamped_vel: false

# joint_broad:
#   ros__parameters:
```

Once we've finished our controller config, we can go back to our URDF and put in that URL. To get the path to work nicely we can use a neat trick to "find" our package and then specify a relative path. Remember to replace `articubot_one` with whatever you've called your package!

```
<parameters>$(find articubot_one)/config/my_controllers.yaml</parameters>
```

Starting the controllers

We can rebuild with colcon to add our new file, then launch our sim just like in the other tutorials. You might notice that the Gazebo window is printing out some slightly different content.

This is a good chance for us to start using the `ros2 control` command line tool. If we type `ros2 control` and hit tab we can see all the options. The only one we'll use right now is `list_hardware_interfaces` and sure enough, we can see our hardware interfaces.

```
dev@dev-PC:~/dev_ws$ ros2 control list_hardware_interfaces
command interfaces
  left_wheel_joint/velocity [unclaimed]
  right_wheel_joint/velocity [unclaimed]
state interfaces
  left_wheel_joint/position
  left_wheel_joint/velocity
  right_wheel_joint/position
  right_wheel_joint/velocity
dev@dev-PC:~/dev_ws$
```

It's tempting to run `list_controllers` too, but (as far as I can tell) this will only work once the controllers have been "loaded" - so let's do that now.

As mentioned earlier, there's a few ways we can do this. We can use the service call (which is fiddly), we can use the `load_controller` command we saw a second ago (which is good, but the syntax changed slightly between foxy and the newer versions, and also it's trickier to use in launch files) or we can use the provided spawner script/node.

Let's try the spawner script.

```
ros2 run controller_manager spawner.py diff_cont
ros2 run controller_manager spawner.py joint_broad
```

This should provide some feedback to let us know it started successfully:

```
newans@pop-desktop:~/dev_ws$ ros2 run controller_manager spawner.py diff_cont
[INFO] [1664360783.679795031] [spawner_diff_cont]: Waiting for /controller_manager services
[INFO] [1664360783.920619634] [spawner_diff_cont]: Loaded diff_cont
[INFO] [1664360783.931330330] [spawner_diff_cont]: Configured and started diff_cont
```

And similarly we should see an indication on the Gazebo tab that it started:

```
[gzserver-2] [INFO] [1664362284.111684757] [gazebo_ros2_control]: Loading gazebo_ros2_control plugin
[gzserver-2] [INFO] [1664362284.114048926] [gazebo_ros2_control]: Starting gazebo_ros2_control plugin in namespace: /
[gzserver-2] [INFO] [1664362284.114084927] [gazebo_ros2_control]: Starting gazebo_ros2_control plugin in ros2 node: gazebo_ros2_control
[gzserver-2] [INFO] [1664362284.114109948] [gazebo_ros2_control]: Loading parameter file /home/newans/dev_ws/install/articubot_one/share/articubot_one/confi
[gzserver-2] [INFO] [1664362284.115848345] [gazebo_ros2_control]: connected to service!! robot_state_publisher
[gzserver-2] [INFO] [1664362284.116356765] [gazebo_ros2_control]: Recieved urdf from param server, parsing...
[gzserver-2] [INFO] [1664362284.123511086] [gazebo_ros2_control]: Loading joint: left_wheel_joint
[gzserver-2] [INFO] [1664362284.123551385] [gazebo_ros2_control]: Command: velocity
[gzserver-2] [INFO] [1664362284.123558613] [gazebo_ros2_control]: State: velocity
[gzserver-2] [INFO] [1664362284.123577862] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1664362284.123584604] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1664362284.123602127] [gazebo_ros2_control]: Loading joint: right_wheel_joint
[gzserver-2] [INFO] [1664362284.123618488] [gazebo_ros2_control]: Command: velocity
[gzserver-2] [INFO] [1664362284.123618457] [gazebo_ros2_control]: State: velocity
[gzserver-2] [INFO] [1664362284.123623116] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1664362284.123627395] [gazebo_ros2_control]: Loading controller manager
[gzserver-2] [INFO] [1664362284.123631742] [gazebo_ros2_control]: Loading gazebo_ros2_control
[gzserver-2] [INFO] [1664362284.123767865] [gazebo_ros2_control]: Loading controller 'diff_cont'
[gzserver-2] [INFO] [1664362284.131867918] [gazebo_ros2_control]: Loading controller 'joint_broad'
[gzserver-2] [INFO] [1664362291.043343442] [controller_manager]: Loading controller 'diff_cont'
[gzserver-2] [INFO] [1664362291.068016615] [controller_manager]: Setting use_sim_time=True for diff_cont to match controller manager (see ros2_control#325 f
[gzserver-2] [INFO] [1664362291.070945323] [controller_manager]: Configuring controller 'diff_cont'
[gzserver-2] [INFO] [1664362296.642679513] [controller_manager]: Loading controller 'joint_broad'
[gzserver-2] [INFO] [1664362296.650767250] [controller_manager]: Setting use_sim_time=True for joint_broad to match controller manager (see ros2_control#325
[gzserver-2] [INFO] [1664362296.660101226] [controller_manager]: Configuring controller 'joint_broad'
```

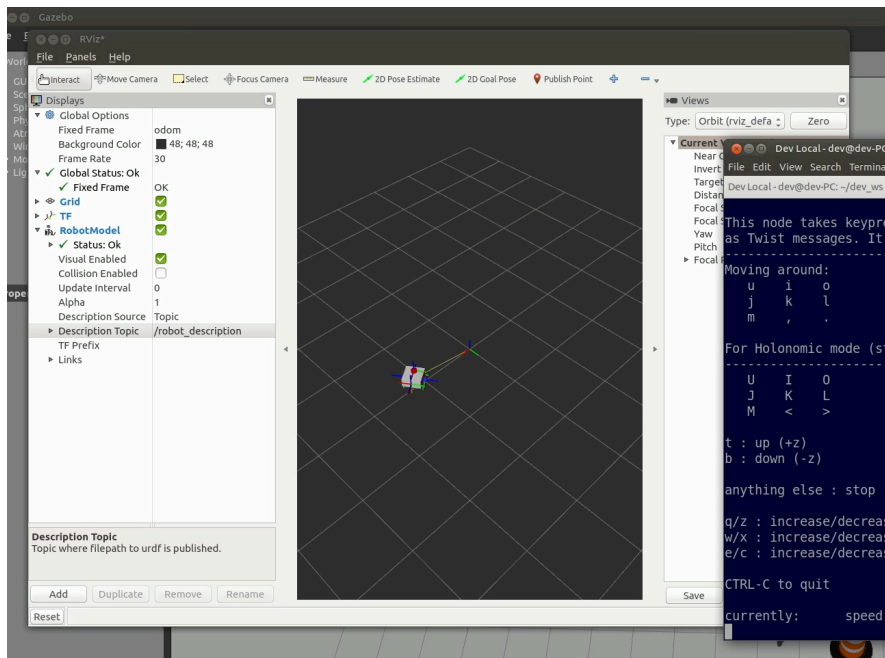
Getting teleop working

We can run `teleop_twist_keyboard` to drive it around, but we need to do it slightly differently this time. Previously, the Gazebo diff drive plugin was listening on `/cmd_vel`, which is what the teleop control was publishing to. The new controller will be listening on `/diff_cont/cmd_vel_unstamped`, so we need to remap the topic with the following command:

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard --ros-args -r /cmd_vel:=/diff_cont/cmd_vel_unstamped
```

Note that if you didn't name your controller `diff_cont` you should substitute it with the name you used. Normally, the `diff_drive_controller` is expecting a `stamped Twist` (velocity) message (`TwistStamped`) which we are disabling with that `use_stamped_vel` parameter. As mentioned earlier, we'll explore this in a future tutorial.

We should now be able to drive our robot around again in Gazebo, and of course we can fire up RViz to monitor everything.



Updating the launch file

Running those spawner scripts each time is a bit of a pain, let's add them to our launch file (`launch_sim.launch.py`) so that they start automatically.

```
diff_drive_spawner = Node(
    package="controller_manager",
    executable="spawner.py",
    arguments=["diff_cont"],
)

joint_broad_spawner = Node(
    package="controller_manager",
    executable="spawner.py",
    arguments=["joint_broad"],
)

# Then at the bottom...

return LaunchDescription([
    rsp,
    gazebo,
    spawn_entity,
    # ... anything else you've added in here...
    diff_drive_spawner,
    joint_broad_spawner
])
```

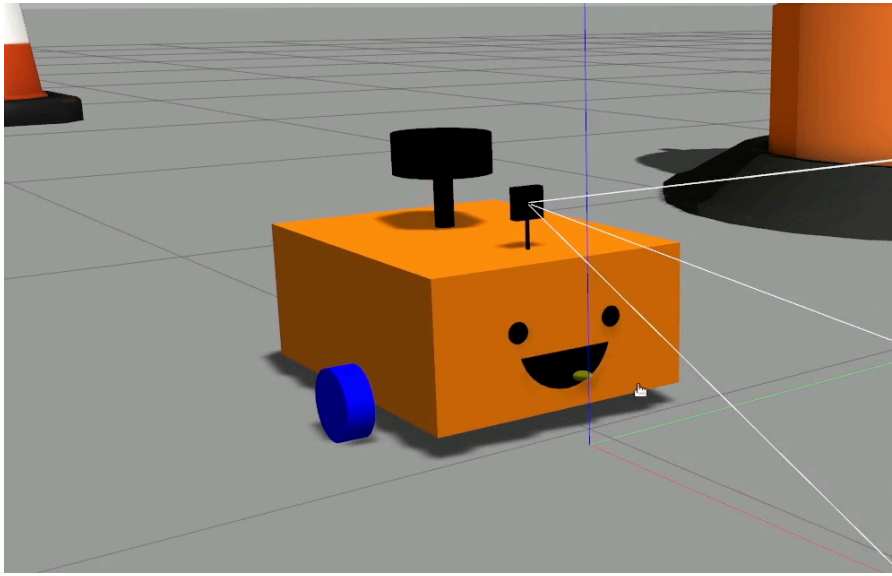
It's possible to set these up on delays to run after the robot spawner has run (see the version on Github for details), or after a timer, but I have found that there is generally a long enough timeout that it is not an issue. The exception to this is sometimes the first time Gazebo is launched after a boot it can take too long and the spawner times out. In this case you may need to kill and rerun it.

Conclusion

If we drive it around now we'll see that it basically works. If you pay close attention though, you'll notice there are a few little things that are not quite right. Before we start trying to use this updated simulation or go ahead with the physical robot, it's worth taking these extra steps. I've put them in [their own post](#) so if you're following along with the build series, you'll want to head there now.

If you were just wanting an overview of `ros2_control`, I hope you found this helpful!

If you have other questions or comments, there is a section below where you can join in the discussion over at the forums.



**EricMack**

Sep '22

Hi Josh just watched the videos you posted dropped for module number 12. I'm getting really excited as I see all of this coming together. I've been collecting the bits and pieces to build my own set up and follow your tutorials.

My question has to do with the physical hardware controller that you used it looks like it's a gaming controller of some kind. I'm not very familiar as I don't have a gaming system or play one.

I realize that ultimately I could use most any gaming controller if I wanna just write my own inner face but for the moment I just wanna duplicate what you've done and then branch off on my own

Do you have any thoughts or recommendations on a particular one to buy and start with So I can use the exact configuration you are using?

[1 reply](#)

**JoshNewans**[▶ EricMack](#) Sep '22

Hey Eric, I'll be diving deeper into that in 2 videos time, but here's a few quick points 😊 :

- I'm using an old wired (USB) Xbox 360 controller
- There are generally 3 types of controllers: USB wired, Bluetooth, and wireless with a USB dongle
 - Wireless is obviously nicer, and means you can connect to either the robot or a development machine on the network, whereas wired kind of restricts you to the latter
 - I don't trust Bluetooth reliability, especially on Linux and on the Pi, so I prefer USB-based options, but many people do have success with Bluetooth (PlayStation controllers being a classic for this). It's also more annoying to re-pair etc.
- Depending on the controller you may or may not need Linux drivers for it (just as if you were playing a game with it). In my limited experience, USB wired ones usually work with no extra software or issues, wireless/Bluetooth can be trickier.
- Once the OS sees it as a "joystick" they are all pretty much the same as far as ROS is concerned. We use `joy/joy_node` to read the raw joystick data from the OS and publish it to `/joy`.
- Now that the joystick input is available as a ROS topic we can do whatever we want with it. For our differential drive robot, we use `teleop_twist_joy` to take those axis/button press measurements from `/joy` and convert them to a Twist message on `/cmd_vel`. The axis/button numbering will be different for different controllers, you just have to set the parameters accordingly (e.g. axis 1 is forward/reverse, axis 2 is left/right, button 4 is enable).
- So with all that being said **pretty much any controller/joystick/gamepad will work**. I personally just like the ergonomics of the Xbox controllers, and I'll see if I can get my wireless one (Xbox One S version w/ Bluetooth) going for the video, otherwise I usually just plug the wired one into a laptop and carry it around.
- In the video I'll also briefly cover connecting a gamepad to your phone, and then connecting the phone to ROS. For that I'll be using [this](#) Bluetooth phone gamepad, but I'd be hesitant to recommend it as I found the joystick deadzones too jarring.
- Thanks for prompting me to think about this. I've just ordered [this one](#) (eBay Australia link) as it seems like it might be a fairly common, cheap one, so hopefully it works ok and I can recommend it in future.

Hope that helps!

**EricMack**

Sep '22

Josh, thank you. Helpful information. I just purchased the same controller for \$13 USD, so I can test what you are doing. I think it is helpful when you post links to what parts you are using so others can follow-along.

-Eric

**EricMack**

Oct '22

Josh, I've been thinking about an idea for a future tutorial for you, related to robot control. It's one I would really like to see.

Once you have the ROS2 Control working with your joystick/game controller, I would like to see you add a control node using an Arduino, LCD shield, a few buttons and potentiometers. This would help us see how your Arduino node would do for I/O, say on a custom portable control pendant. Here's what I have in mind:

INPUTS

2 buttons - Honk Remote buzzer or flash LED headlights.

3 potentiometers - speed control, throttle (and, ideally, steering, perhaps using a dual pot joystick but I understand that is more involved).

OUTPUTS

Buzzer - when a bump switch on robot is activated it should trigger this buzzer/light/whatever

LCD - One line for Speed and heading (relative to anything you can grab); One line for angle of something, like the wheels; and One line for distance to object – whatever is in front.

What I am really keen to learn is how you would plan this and do the interface to both broadcast and listen to the appropriate pins/signals.

The concepts you develop here, perhaps with input from the community, would allow us to make custom controllers with bidirectional capabilities.

If you do not have an LCD shield, I would be happy to make a contribution to cover one.

I would really like to see this in a future video.

Thanks for all you do to teach and guide us.

Eric

Update: If the above is a task that is better suited for a Raspberry Pi, that is OK. If so, I would like to understand why (e.g. software, IO, libraries, etc. If Raspberry Pi, could we use a PICO W with an I2C Shield or something?). This is what I think an excellent concept that people [like me] could run with once they have an understanding of how to approach this. Thanks for considering.

[1 reply](#)

**JoshNewans**[▶ EricMack](#) Oct '22

Hey Eric, interesting idea!

Are you imagining something like the ones James Bruton has made for his videos?

He's had a few different ones (at least 3), but the "old one" shown at the start of [this video](#) seems to be what you're describing.

I had not considered it, but that's not a bad idea, I will give it further thought. I'd probably be leaning toward the Pico W (as you've suggested) and programming it with micro-ROS or CircuitPython, but those decisions would mostly be driven by me having never used those platforms before and it seems like a good starter project to get my feet wet.

[1 reply](#)

**EricMack**[▶ JoshNewans](#) Oct '22

Yes! I had forgotten about his video. I've had the idea in the back of my mind for too many years, but the missing link was the software. You are convincing me that ROS2 is the way to go.

Please do consider the basic proposal in one of your upcoming videos. Happy to help brainstorm.

UPDATE: I just watched the video again. I like his idea of using a touch screen for more virtual buttons. Might be cool someday to touch a map of the room and the robot goes there. Then a PTZ Camera on the robot can beam video back. He uses an Arduino to manage the I/O and send/receive commands via Serial to USB into the Pi, which runs the ROS node. Source code in his github but I think it may be ROS not ROS2.

**Niranjn_Rao**

Nov '22

Hi Josh,

Thanks for share this ros2 control video it is very helpful.

I am getting this error while running "ros2 launch my_bot launch_sim.launch.py". If you have any suggestions to resolve this issue. kindly pleas share it with me.

[gzserver-2] [ERROR] [1669334236.428722199] [gazebo_ros2_control]: parser error Couldn't parse params file: '-params-file /home/niranjan/dev_ws/install/my_bot/share/my_bot/config/my_control_params.yaml'. Error: Cannot have a value before ros__parameters at line 4, at /tmp/binarydeb/ros-foxy-rcl-yaml-param-parser-1.1.14/src/parse.c:623, at /tmp/binarydeb/ros-foxy-rcl-1.1.14/src/rcl/arguments.c:388

Thanks
Niranjan

[1 reply](#)



Niranjan_Rao

[► Niranjan_Rao](#) Nov '22

Hi Josh,

I think I am able to resolve this issue. It looks like the my_controls.yml parameter file ros__parameters requires two underscore I had a typo here. After fixing the typo the issue got resolved.

Thanks
Niranjan

[1 reply](#)



JoshNewans

[► Niranjan_Rao](#) Nov '22

Glad you were able to get it fixed!
And yes that double underscore is an easy mistake to make, I've definitely tripped on that before!

[1 reply](#)



Stress_Robotics

Dec '22

Hi I have a problem guys, after implementing ros2_control , the robot moves in the opposite direction (forward and backwards) in rviz2... Gazebo is working fine though (except for the publish_rate thingy)... please any ideas on how to solve this?

[2 replies](#)



JoshNewans

[► Stress_Robotics](#) Jan '23

Hmm, the first thing I'd say would be to step through and make sure you understand your kinematic chain, i.e. which axes are pointing in which direction. I believe I had Z pointing out, i.e. the left wheel is "positive-Z-rotation-forwards" and the right wheel is "positive-z-rotation-backwards".

The axis of the right wheel joint is then flipped, and you should be able to check this by not running ros2_control and instead using joint_state_publisher to manually control the joint angles. Make sure that a positive joint corresponds to the robot moving forward and vice versa.

Then in ros2_control1.xacro and my_controllers.yaml (or whatever you have called them), ensure that left and right are assigned correctly.

Finally, make sure that your fixed frame is odom in RViz.

Let me know how you go!

[2 replies](#)



mhered

[► JoshNewans](#) Feb '23

Hello Josh,

After implementing the fix to increase gazebo clock rate, I get the following error message when running gazebo - even though it otherwise seems to be working fine:

```
...
[gzserver-2] Error [parser.cc:403] Error parsing XML in file
[/home/mhered/dev_ws/install/manolobot_uno/share/manolobot_uno/config/gazebo_params.yaml]: Error document empty.
...
```

It appears it is trying to interpret the gazebo_params.yaml file as XML and failing.

I noticed also your comment in the blog post:

On my computer I also get an error printed in the Gazebo terminal which is a bit annoying, I think the Gazebo command line parser is treating the file as an extra SDF to load and failing. We can just ignore it, but if you've got a fix then let me know!

Is this the error you refer to?



Stress_Robotics

[► JoshNewans](#) Feb '23

I got it fixed. it was the kinematic chain problem. Thank you

[1 reply](#)



tiptonnet

Mar '23

So I've been following your tutorial for a project. Thank you BTW. It's been hard to find Raspberry Pi examples. Problem is. I ran this part of the tutorial last night and it all seemed to work with exception of a type in my config. The robot started doing flips when set to move forward. Anyways I think I got that solved but now, today when I try to run it the controller manager is not loading... All the sudden it's just not there any more.

Output:

```
ros2 run controller_manager spawner diff_cont
[INFO] [1679414845.079311127] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[INFO] [1679414847.090650840] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[INFO] [1679414849.100639643] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[INFO] [1679414851.114171358] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[ERROR] [1679414853.122073734] [spawner_diff_cont]: Controller manager not available
```

Why now? Worked last night. I can't even list interfaces... Same problem.

[1 reply](#)



tiptonnet

Mar '23

Okay... Somehow controller manager mysteriously started working. Thanks to Niranjan_Rao pointing out the underscore issue, that did make a difference.

But now he is back to doing flips instead of moving forward or reverse. I'm sure I'll figure it out, obviously a config issue.



Afshan_Navazdeen

Apr '23

Hi Josh,
Didn't find any issues until Ros2_control was introduced.

With gazebo_ros2_control my robot moves only forward and backwards, doesn't respond the angular velocity command and the /odom frame doesn't show up in the RVIZ fixed frame selection list.

I have tried it on Foxy and humble. the result is the same.

Any ideas how I can fix this.

[2 replies](#)



baolongVietNam

May '23

B	JoshNewans Hi Josh, Your tutorial is very helpful. and i want to use ros_control to control my robot with ackermann type instead of your diff. You can make a video tutorial. I sincerely thank	▶	Afshan_Navazdeen	May '23
A	Afshan_Navazdeen Further into the investigation i have noticed the odom TF has the below error. Error: TF NAN_INPUT: Ignoring transform for child_frame_id "base_link" from authority "Authority undetectable" because of a nan value in the transform (-nan -nan 0.000000) (-nan -nan -nan -nan) at line 278 in /tmp/binarydeb/ros-foxy-tf2-0.13.14/src/buffer_core.cpp Error: TF DENORMALIZED_QUATERNION: Ignoring transform for child_frame_id "base_link" from authority "Authority undetectable" because of an invalid quaternion in the transform (-nan -nan -nan -nan) @JoshNewans Can you please help me out on what causes this error and how to resolve it. This has practically stopped me in continuing further in to this wonderful tutorial. 1 reply	▶	Afshan_Navazdeen	Jun '23
A	Alessandro_Gomez I'm having the same problem. Everything works except the angular velocity 1 reply	▶	Afshan_Navazdeen	Jun '23
Z	Ziad_Shoeib Hello, I have been following along and thanks for your great efforts, everything had been working fine with me until this point. When I tried the laser scan at rviz with ros2_control, the scan moves totally with the robot, the dots move with the orientation of the robot not just wiggle as the small problem but it totally moves. When I switch to gazebo_control everything comes back to normal, not sure what the problem can be? 1 reply			Jun '23
A	Afshan_Navazdeen @Alessandro_Gomez Did you find a solution to this?	▶	Alessandro_Gomez	Jun '23
V	vijay_sankar I am also facing the same issue. ros2 launch articubot_one launch_sim.launch.py everything is working fine. when i launch ros2 launch articubot_one launch_robot.launch.py i am having controller_manager issues . I did not connect my joystick. and my physical robot has no response.	▶	tiptonnet	Jun '23
P	Phyo_Min_Thant FYI. In ROS2 Humble, I could perfectly run with just "spawner" not "spawner.py". It is a little change as you mentioned. 2 replies			Jul '23
V	vijay_sankar How did you fix the Hardware_interface issues in Humble using GitHub - joshnewans/diffdrive_arduino: A simple ROS Control Hardware Interface for differential drive control of robot whose motors are connected via an Arduino (or similar microcontroller) . this Diffdrive_arduino . I was stuck here.	▶	Phyo_Min_Thant	Jul '23
U	Uby Hi, I facing some issue with gazebo ros2 control package. I followed the instruction for the ros2 control and launch the launch_sim.launch.py everything work perfect but could not connect with gazebo_ros2_control The error msg: [gzserver-2] [ERROR] [1691414356.585613673] [gazebo_ros2_control]: Error parsing URDF in gazebo_ros2_control plugin, plugin not active : text not specified in the min tag because this I wasn't able to connect with hardware interface error msg: ros2 control list hardware_interfaces Could not contact service /controller_manager/list_hardware_interfaces Can you guys help me to figure out what mistake I have made. Thank you for your help. 2 replies			Aug '23
E	Evanston_Garneau Hey Josh, I am having trouble running the command ros2 run controller_manager spawner.py diff_cont it is saying that there is no executable found. I have installed and done everything the same as you up until that point. Any suggestions? From Evanston 3 replies	▶	Afshan_Navazdeen	Aug '23
U	Uby I figure it out the problem. It was my linux system i just reinstall it an now it working fine	▶	Uby	Aug '23
U	Uby Hello, I am driving the robot in gazebo with teleop twist keyboard but it not moving in rviz.			Aug '23
P	phil123456 was working fine launching the controllers manually but with the launcher spawner.py-5] [INFO] [1695724540.948537156] [spawner_diff_cont]: Waiting for /controller_manager services [spawner.py-6] [INFO] [1695724541.043151151] [spawner_joint_broad]: Waiting for /controller_manager services [gzclient -3] context mismatch in svg_surface_destroy [gzclient -3] context mismatch in svg_surface_destroy [spawner.py-5] [INFO] [1695724542.965812070] [spawner_diff_cont]: Waiting for /controller_manager services [spawner.py-6] [INFO] [1695724543.056949421] [spawner_joint_broad]: Waiting for /controller_manager services [spawner.py-5] [INFO] [1695724544.983464604] [spawner_diff_cont]: Waiting for /controller_manager services [spawner.py-6] [INFO] [1695724545.078081205] [spawner_joint_broad]: Waiting for /controller_manager services [spawner.py-5] [INFO] [1695724546.997859933] [spawner_diff_cont]: Waiting for /controller_manager services [spawner.py-6] [INFO] [1695724547.087982065] [spawner_joint_broad]: Waiting for /controller_manager services [spawner.py-5] [ERROR] [1695724549.01000553] [spawner_diff_cont]: Controller manager not available			Sep '23



Nguyen Van Vy

Nov '20

Hi there, I am very thankful for your robot tutorial. Can you let me ask if skipping the tutorial section in video 12a will have much impact on subsequent tutorials. I don't really need great precision



Mohammed_Thabrez

► Evanston_Garneau Nov '23

Can anyone help out with this problem . I am also facing the same issue

1 reply



Mohammed_Thabrez

Nov '23

@JoshNewans can u help me with this

I download the entire articubot_one code from your Github and am running the command - ros2 launch articubot_one launch_sim.launch.py world:=src/articubot_one/worlds/obstacles.sdf

```
thabrez@thabrez-Inspiron-15-7000-Gaming:~/dev_ws2$ ros2 launch articubot_one launch_sim.launch.py
world:=src/articubot_one/worlds/obstacles.sdf
[INFO] [launch]: All log files can be found below /home/thabrez/.ros/log/2023-11-21-00-51-30-748484-thabrez-Inspiron-15-7000-Gaming-7568
[INFO] [launch]: Default logging verbosity is set to INFO
[ERROR] [launch]: Caught exception in launch (see debug for traceback): executable 'spawner.py' not found on the libexec directory
'/opt/ros/iron/lib/controller_manager'
```

I checked many times . used source install/setup.bash , correct directory too .
I have ros iron controller manager installed
The spawner.py file also exists in location opt/ros/iron/lib/controller_manager
Have done no modification to original code
newworld.sdf
Can u help me solve this ?? please

1 reply



mfekjaer

► Mohammed_Thabrez Nov '23

Hi @Mohammed_Thabrez

Looks like you are using ros2 iron, this tutorial is for foxy, but it is updated to humble, see https://www.youtube.com/watch?v=qoj5_fVBPII for the update

Try the humble branches of the github repositories.



hoonle135

► Mohammed_Thabrez Dec '23

Hi @JoshNewans

I'm currently facing some problems while working with ROS2 Foxy. I've followed the installation steps by executing the command:

```
sudo apt install ros-foxy-ros2-control ros-foxy-ros2-controllers ros-foxy-gazebo-ros2-control
```

However, when I attempt to launch the simulation using:

```
ros2 launch hoonle_bot launch_sim.launch.py world:=./src/hoonle_bot/worlds/obstacles.world
```

I encounter the following error:

```
[gzserver-2] [WARN] [1702871480.663104398] [gazebo_ros2_control]: Desired controller update period (0.0333333 s) is
slower than the gazebo simulation period (0.001 s).
[gzserver-2] [INFO] [1702871480.665948117] [gazebo_ros2_control]: Loaded gazebo_ros2_control.
[gzclient -3] context mismatch in svga_surface_destroy
[gzclient -3] context mismatch in svga_surface_destroy
[gzserver-2] [INFO] [1702871590.487554188] [controller_manager]: Loading controller 'diff_cont'
[gzserver-2] [INFO] [1702871590.717861782] [controller_manager]: Setting use_sim_time=True for diff_cont to match
controller manager (see ros2_control#325 for details)
[gzserver-2] [INFO] [1702871590.875489982] [controller_manager]: Configuring controller 'diff_cont'
[gzserver-2] [INFO] [1702871724.122437887] [controller_manager]: Configuring controller 'diff_cont'
[gzserver-2] [ERROR] [1702871724.122766325] [controller_manager]: Controller 'diff_cont' can not be configured from
'active' state.
```

Additionally, when I try to execute ros2 run control, I encounter the following error:

```
usage: ros2 run [-h] [--prefix PREFIX] package_name executable_name ...
ros2 run: error: the following arguments are required: executable_name, argv
```

Could you kindly assist me in resolving these issues?

Thank you for your help.

1 reply



Mohammed_Thabrez

► hoonle135 Dec '23

I FINALLY AND SUCCESSFULLY SOLVED ALL MY ISSUES AND WAS ABLE TO EXECUTE SLAM AND NAV2 ON BOTH SIMULATOR AND ALSO ON A REAL ROBOT THAT I BUILT .

I used ros Iron cause Foxy has reached its end of life (eol).

At the start of my project itself I was not able to download ros-foxy packages and i found that its dead .
check these :

- if u are actually able to download ros-foxy packages
- check for existence of ros2 FOXY
- check for gazebo and its packages
- try changing obstacles.world to a new obstacles2.sdf file (new extension from world to sdf)
- try running installation of controller packages command again
- try colcon build with symlink again
- download code directly from github rather than typing yourself

without the controller working u will get error in ros2 run control

1 reply



stevej

► Mohammed_Thabrez Dec '23

Hello Mohammed could you list the libraries and dependencies and drivers used to get your simulated robot to work as well as the physical robot...
thank you and congrats

1 reply



Mohammed_Thabrez

► stevej Dec '23

Sorry i didnt make a record of every package i downloaded .

I completely followed the articulated bot playlist and also watched the last video in the playlist that talks about Humble changes too .



WaelSonic

► Phyo_Min_Thant Dec '23

Hey man thank you so much... It took me 4 hours debugging until I read your comment. You're genius man. Can I know how did you figured that out?



WaelSonics

you should remove the .py from spawner.py.
Check the update video in the series (Are you ready to Humble...)

► Evanston_Garneau Dec '23



jonnyr

Try:

```
ros2 run controller_manager spawner -h  
  
ros2 run controller_manager spawner diff_cont  
  
No .py
```

► Evanston_Garneau Jan '24



Umberto

Hi Josh,

Jan '24

@Josh: As many have said, much thanks for your great tutorials on ROS!

I'm building a robot based on your Articubot robot using 2 motors and a (ultrasound) sonar sensor (HCSR04). I am able to control the motors and drive around. But I'm stuck on how to correctly integrate the sonar sensor into ROS.

I've integrated the sonar sensor in joshnewans/ros_arduino_bridge example exposing distance information via the serial interface (via a "h" command, the distance as reported by the sonar sensor is returned)

Now I'm trying to create a hardware interface that publishes this sonar distance information in a ROS Topic sensor_msg/Range message. I followed the tutorial '12. Solving the problem EVERY robot has (with ros2_control)' and concluded that I need to create another State Interface (see <https://articulatedrobotics.xyz/media/assets/posts/mobile-robot/12-ros2-control/controllers.png>)

I tried to update joshnewans/diffdrive_arduino example to publishing the sensor_msg/Range message in diffdrive_arduino.cpp. But I am unable to do this since I cannot access ROS publish functions. Also reading further on the concepts of ROS, it seems not the correct way to do since diffdrive_arduino is for controlling the diff drive motors and NOT exposing a sonar sensor.

Then I tried to create another node that reads the serial information and publishes the Range message. But then I have the problem that you cannot use 1 serial interface by 2 nodes that each run as separate process.

So my question is:

How do you create 2 controllers exposing 2 hardware_interfaces (System & Sensor) that uses the same serial interface? Is there a way to share 1 serial interface between ROS nodes?

Regards,

Huub



nisat_av

Hi Josh,

At first, your tutorials are helping in a great course for my career and thanks for your tutorials.

Feb '24

I am trying to understand this ros2_control thing. but I am having the following issue when I am trying to load the controllers. btw I am using ros2-humble distro.

```
nisat@nt:~/mangobee_robot$ ros2 run controller_manager spawner diff_cont  
[INFO] [1706769460.539068927] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist  
[INFO] [1706769462.551452191] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist  
[INFO] [1706769464.564305671] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist  
[INFO] [1706769466.575603025] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist  
[ERROR] [1706769468.585726734] [spawner_diff_cont]: Controller manager not available  
[ros2run]: Process exited with failure 1
```

my_controllers.yaml file as follows:

```
controller_manager:  
  ros__parameters:  
    update_rate: 30  
    use_sim_time: true  
  
  diff_cont:  
    type: diff_drive_controller/DiffDriveController  
  
  joint_broad:  
    type: joint_state_broadcaster/JointStateBroadcaster
```

```
diff_cont:  
  ros__parameters:  
  
    publish_rate: 100.0  
    base_frame_id: base_link  
  
    left_wheel_names: ['left_wheel_joint']  
    right_wheel_names: ['right_wheel_joint']  
    wheel_separation: 0.35  
    wheel_radius: 0.05  
  
    use_stamped_vel: false
```

ros2_control.xacro file as follows,

```
<?xml version="1.0"?>  
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">  
  
  <ros2_control name="GazeboSystem" type="system">  
    <hardware>  
      <plugin>gazebo_ros2_control/GazeboSystem</plugin>  
    </hardware>  
  
    <joint name="left_wheel_joint">  
      <command_interface name="velocity">  
        <param name="min">-10</param>  
        <param name="max">10</param>  
      </command_interface>  
      <state_interface name="position" />  
      <state_interface name="velocity" />  
    </joint>  
  
    <joint name="right_wheel_joint">  
      <command_interface name="velocity">  
        <param name="min">-10</param>  
        <param name="max">10</param>  
      </command_interface>  
      <state_interface name="position" />  
      <state_interface name="velocity" />  
    </joint>  
  </ros2_control>  
</robot>
```

```
</command_interface>
<state_interface name="velocity" />
<state_interface name="position" />
</joint>

<gazebo>
  <plugin filename="libgazebo_ros2_control.so" name="gazebo_ros2_control">
    <parameters>$(find Mangobee_DifferentialDriveWheel_Robot)/config/my_controllers.yaml</parameters>
  </plugin>
</gazebo>

</ros2_control>

</robot>
```

Please help me solve this issue.

Thank you
Nisath



cyberbilly

Feb '24

I am using ros2 iron and have "Contorller manager not available" errors. To make sure its not something I did, I downloaded articubot_one and it has the same issue.

First i fixed the .py issue to get this far:

```
diff_drive_spawner = Node(
    package="controller_manager",
    executable="spawner",
    arguments=["diff_cont"],
)

joint_broad_spawner = Node(
    package="controller_manager",
    executable="spawner",
    arguments=["joint_broad"],
)
```

Now when I enter

```
ros2 launch articubot_one launch_sim.launch.py world:=./src/my_bot/worlds/obstacles.world
```

Looks like I get two errors:

```
[ERROR] [gzserver-4]: process has died [pid 35946, exit code 255, cmd 'gzserver ./src/my_bot/worlds/obstacles.world -
slibgazebo_ros_init.so -slibgazebo_ros_factory.so -slibgazebo_ros_force_system.so --ros-args --params-file
/home/willli/shared/dev_ws/install/articubot_one/share/articubot_one/config/gazebo_params.yaml'].
```

Either the result or the real problem is the following:

```
[spawn_entity.py-4] [INFO] [1706795144.454419682] [spawn_entity]: Waiting for entity xml on robot_description
[spawn_entity.py-4] [INFO] [1706795144.465585204] [spawn_entity]: Waiting for service /spawn_entity, timeout = 30
[spawn_entity.py-4] [INFO] [1706795144.465941557] [spawn_entity]: Waiting for service /spawn_entity
[spawner-6] [INFO] [1706795146.204682358] [spawner_joint_broad]: Waiting for '/controller_manager' node to exist
[spawner-5] [INFO] [1706795146.405621127] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[spawner-6] [INFO] [1706795148.218729921] [spawner_joint_broad]: Waiting for '/controller_manager' node to exist
[spawner-5] [INFO] [1706795148.418687173] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[spawner-6] [INFO] [1706795150.234781109] [spawner_joint_broad]: Waiting for '/controller_manager' node to exist
[spawner-5] [INFO] [1706795150.429329085] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[spawner-6] [INFO] [1706795152.248062370] [spawner_joint_broad]: Waiting for '/controller_manager' node to exist
[spawner-5] [INFO] [1706795152.447229883] [spawner_diff_cont]: Waiting for '/controller_manager' node to exist
[spawner-6] [ERROR] [1706795154.261474129] [spawner_joint_broad]: Controller manager not available
[spawner-5] [ERROR] [1706795154.459666959] [spawner_diff_cont]: Controller manager not available
[ERROR] [spawner-6]: process has died [pid 34051, exit code 1, cmd '/opt/ros/iron/lib/controller_manager/spawner
joint_broad --ros-args'].
[ERROR] [spawner-5]: process has died [pid 34042, exit code 1, cmd '/opt/ros/iron/lib/controller_manager/spawner
diff_cont --ros-args'].
[spawn_entity.py-4] [ERROR] [1706795174.516724938] [spawn_entity]: Service %s/spawn_entity unavailable. Was Gazebo
started with GazeboRosFactory?
[spawn_entity.py-4] [ERROR] [1706795174.517468743] [spawn_entity]: Spawn service failed. Exiting.
```

help

[1 reply](#)



cyberbilly

[► cyberbilly](#) Feb '24

So... I couldn't get past this error smurfing the innernut yielded notta. I then blew my package away and downloaded joshes to the same result... I then download an unrelated Ros2 to package from some other unrelated github and YIKES SAME same... Then I completely removed ros2-iron and installed ros2-humble and now everything is fine. It all works as specified. There is some issue/change in iron



Mohit Gupta

[► Ziad_Shoeib](#) Mar '24

Hi Ziad, I am facing the same issue. Did you find the solution?



Hasan Basari

[► Stress_Robotics](#) Apr '24

how would you solve this issue please explain clearly



Hasan Basari

[► Stress_Robotics](#) Apr '24

you can solve this issues, how would you solve this issue please explain clearly



saad_waqar

Apr '24

Hi [@JoshNewans](#)

Thank you so much for the hardwork.

I have been following and developing a physical robot as well.

I am facing an issue that:

When I shifted to ros2 control, the robot is driving in the opposite direction (in the Gazebo simulation), but when I was working with Gazebo diff_drive, it was running perfectly fine. Can anyone please guide me? I have been struggling with this for 2 days now.

I checked your response to another similar question, so I have similarly used the axis orientations for the wheels, it works good with joint_state_publisher and when working with gazebo_diff_drive, but when I use ros2 control, the wheels rotate inversely.

Can you please shed some light?

Thank you so much!

Thank you



Tim Hayes

[► JoshNewans](#) May '24

Hi [@saad_waqar](#), have you been able to solve the problem and had to look back at the [robot](#) should know the same as the previous code



baha_jouli

Jun '24

To change "l298" to "bts,7960b" what modifications are needed in the code?



ThurzinRB

Uby Jul '24

I don't know if it's the same issue. I was having the same error message: "Could not contact service /controller_manager/list_hardware_interfaces."

After some troubleshooting, I found that there was a mistake in my ros2_control.xacro file:

I had put the <gazebo> tag with its plugin inside the <ros2_control> tag when it needed to be placed separately.



Flamethrower

Aug '24

@JoshNewans Hello Josh. Thank you for all these tutorials. You may be aware of the issue where the controller_manager is apparently not available. Specifically, the output when launching is this:

```
[INFO] [launch]: All log files can be found below /home/flamethrower/.ros/log/2024-08-04-18-35-28-656714-robot-3619
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [robot_state_publisher-1]: process started with pid [3621]
[INFO] [gzserver-2]: process started with pid [3623]
[INFO] [gzclient -3]: process started with pid [3625]
[INFO] [spawn_entity.py-4]: process started with pid [3628]
[INFO] [spawner.py-5]: process started with pid [3631]
[INFO] [spawner.py-6]: process started with pid [3633]
[robot_state_publisher-1] Parsing robot urdf xml string.
[robot_state_publisher-1] Link FL_wheel had 0 children
[robot_state_publisher-1] Link FR_wheel had 0 children
[robot_state_publisher-1] Link RL_wheel had 0 children
[robot_state_publisher-1] Link RR_wheel had 0 children
[robot_state_publisher-1] Link chassis had 1 children
[robot_state_publisher-1] Link laser_frame had 0 children
[robot_state_publisher-1] [INFO] [1722789331.093667060] [robot_state_publisher]: got segment FL_wheel
[robot_state_publisher-1] [INFO] [1722789331.098568099] [robot_state_publisher]: got segment FR_wheel
[robot_state_publisher-1] [INFO] [1722789331.098785080] [robot_state_publisher]: got segment RL_wheel
[robot_state_publisher-1] [INFO] [1722789331.098873561] [robot_state_publisher]: got segment RR_wheel
[robot_state_publisher-1] [INFO] [1722789331.098906653] [robot_state_publisher]: got segment base_link
[robot_state_publisher-1] [INFO] [1722789331.098934339] [robot_state_publisher]: got segment chassis
[robot_state_publisher-1] [INFO] [1722789331.098960764] [robot_state_publisher]: got segment laser_frame
[spawner.py-5] [INFO] [1722789332.642764464] [spawner_diff_cont]: Waiting for /controller_manager services
[spawner.py-6] [INFO] [1722789332.753680821] [spawner_joint_broad]: Waiting for /controller_manager services
[spawn_entity.py-4] [INFO] [1722789333.814943066] [spawn_entity]: Spawn Entity started
[spawn_entity.py-4] [INFO] [1722789333.817251300] [spawn_entity]: Loading entity published on topic robot_description
[spawn_entity.py-4] [INFO] [1722789333.824613498] [spawn_entity]: Waiting for entity xml on robot_description
[spawn_entity.py-4] [INFO] [1722789333.840584852] [spawn_entity]: Waiting for service /spawn_entity, timeout = 30
[spawn_entity.py-4] [INFO] [1722789333.842771012] [spawn_entity]: Waiting for service /spawn_entity
[spawner.py-5] [INFO] [1722789334.688257733] [spawner_diff_cont]: Waiting for /controller_manager services
[spawner.py-6] [INFO] [1722789334.793537609] [spawner_joint_broad]: Waiting for /controller_manager services
[spawner.py-5] [INFO] [1722789336.754853755] [spawner_diff_cont]: Waiting for /controller_manager services
[spawner.py-6] [INFO] [1722789336.890463941] [spawner_joint_broad]: Waiting for /controller_manager services
[spawner.py-5] [INFO] [1722789338.799318653] [spawner_diff_cont]: Waiting for /controller_manager services
[spawner.py-6] [INFO] [1722789338.942930759] [spawner_joint_broad]: Waiting for /controller_manager services
[spawn_entity.py-4] [INFO] [1722789339.373300524] [spawn_entity]: Calling service /spawn_entity
[spawn_entity.py-4] [INFO] [1722789340.397202490] [spawn_entity]: Spawn status: SpawnEntity: Successfully spawned entity [robot]
[gzserver-2] [WARN] [1722789340.508036852] [rcl]: Found remap rule '~/out:=scan'. This syntax is deprecated. Use '--ros-args --remap ~/out:=scan' instead.
[gzserver-2] [WARN] [1722789340.586504745] [rcl]: Found remap rule '~/out:=scan'. This syntax is deprecated. Use '--ros-args --remap ~/out:=scan' instead.
[INFO] [spawn_entity.py-4]: process has finished cleanly [pid 3628]
[gzserver-2] [INFO] [1722789340.836605840] [gazebo_ros2_control]: Loading gazebo_ros2_control plugin
[gzserver-2] [INFO] [1722789340.856579331] [gazebo_ros2_control]: Starting gazebo_ros2_control plugin in namespace: /
[gzserver-2] [INFO] [1722789340.860006190] [gazebo_ros2_control]: Starting gazebo_ros2_control plugin in ros 2 node: gazebo_ros2_control
[gzserver-2] [INFO] [1722789340.860619318] [gazebo_ros2_control]: Loading parameter files
/home/flamethrower/robot_ws/install/robot/share/robot/config/my_controllers.yaml
[spawner.py-5] [INFO] [1722789340.877581837] [spawner_diff_cont]: Waiting for /controller_manager services
[gzserver-2] [INFO] [1722789340.884953905] [gazebo_ros2_control]: connected to service!! robot_state_publisher
[gzserver-2] [INFO] [1722789340.893442322] [gazebo_ros2_control]: Recieved urdf from param server, parsing...
[gzserver-2] [INFO] [1722789340.948132774] [gazebo_ros2_control]: Loading joint: FL_wheel_joint
[gzserver-2] [INFO] [1722789340.948472162] [gazebo_ros2_control]: State:
[gzserver-2] [INFO] [1722789340.948567995] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.948771920] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1722789340.948853605] [gazebo_ros2_control]: Command:
[gzserver-2] [INFO] [1722789340.948888198] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949095438] [gazebo_ros2_control]: Loading joint: FR_wheel_joint
[gzserver-2] [INFO] [1722789340.949171771] [gazebo_ros2_control]: State:
[gzserver-2] [INFO] [1722789340.949207993] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949243345] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1722789340.949275752] [gazebo_ros2_control]: Command:
[gzserver-2] [INFO] [1722789340.949304863] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949344659] [gazebo_ros2_control]: Loading joint: RL_wheel_joint
[gzserver-2] [INFO] [1722789340.949382381] [gazebo_ros2_control]: State:
[gzserver-2] [INFO] [1722789340.949414789] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949445659] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1722789340.949479307] [gazebo_ros2_control]: Command:
[gzserver-2] [INFO] [1722789340.949507511] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949543029] [gazebo_ros2_control]: Loading joint: RR_wheel_joint
[gzserver-2] [INFO] [1722789340.949577140] [gazebo_ros2_control]: State:
[gzserver-2] [INFO] [1722789340.949608121] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.949638640] [gazebo_ros2_control]: position
[gzserver-2] [INFO] [1722789340.949670269] [gazebo_ros2_control]: Command:
[gzserver-2] [INFO] [1722789340.949699621] [gazebo_ros2_control]: velocity
[gzserver-2] [INFO] [1722789340.951049691] [gazebo_ros2_control]: Loading controller_manager
[gzserver-2] terminate called after throwing an instance of 'std::system_error'
[gzserver-2] what(): Invalid argument
[gzserver-2] Aborted (core dumped)
[ERROR] [gzserver-2]: process has died [pid 3623, exit code 134, cmd 'gzserver -s libgazebo_ros_init.so -s libgazebo_ros_factory.so -s libgazebo_ros_force_system.so '].
```

I have tested this by downloading the project folder off of GitHub and the error persists. I'm pretty sure I've downloaded all of the necessary packages, so what might be the problem here? Thanks in advance.

1 reply



george_bethel

Sep '24



Flamethrower:

Waiting

What ROS2 distro are you using?

Also, there seem to be an issue with starting the controller manager



AlePuglisi

Dec '24

Hey Josh, I love this series, It helped me learn ROS 2 with practical examples.

At the end of the ros2_control video, you introduce transmissions, as I understand, they are useful to set up a gearbox interface for the simulator. I'm working on a legged robot simulation and control with Gazebo Classic and ros2 humble, with controllers based on ros2_control.

I need to simulate the motor gearbox to make the simulation more reliable.

Do you know how to set up a reduction ratio properly? All the resources I found online seem draft and even the project that uses it is not working properly, setting a reduction ratio of 0.0 or a string as ratio is fine for the parser (it is completely ignored...).

Do you have some resources, examples or suggestions?



Golden

4 May

yo,

so I have following this tutorial till this part so far (video 12) and there is one issue that I am facing for a long time now with no any solution,

the issue is, when I run the gazebo simulation after adding the codes, the robot with the slightest touch when I am running telop_twist_keyboards, it just go ham, it moves there is no tomorrow, no control nothing, and it rotates like a washing machine without stopping in between, I played around with the yaml controllers but no change, I tried to send the vel instead by publishing the topic, and it still run ham, there is no change in the code from the original code, yet for some reason mine is crazy. I figured the spawn instead of spawn.py but there is nothing much so far.

I am using ros2 humble and ubuntu 22

this is the current ros2 plugin code

```
<ros2_control name="GazeboSystem" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>
  <joint name="left_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
  <joint name="right_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
</ros2_control>

<gazebo>
  <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
    <parameters>home/ubuntu/ros2_ws/src/articubot_one/my_controllers.yaml</parameters>
  </plugin>
</gazebo>
```

and this is the current .yaml controller

```
controller_manager:
  ros__parameters:
    update_rate: 30
    use_sim_time: true

diff_cont:
  type: diff_drive_controller/DiffDriveController

joint_broad:
  type: joint_state_broadcaster/JointStateBroadcaster

diff_cont:
  ros__parameters:

publish_rate: 30.0

base_frame_id: base_link

left_wheel_names: ['left_wheel_joint']
right_wheel_names: ['right_wheel_joint']
wheel_separation: 0.35
wheel_radius: 0.05

use_stamped_vel: false
```

please help me with this issue, anything or any clue will be helpful.

[Continue Discussion](#)



